C++20 Ranges and Views - Modern Iteration and Transformation

Theory, Edge Cases & Code Examples

- ✓ The Problem They Solve
- ✓ The Type Hierarchy
- ✓ Lazy, Non-Owning Range Adaptors
- ✓ The `|` Operator Magic
- ✓ Modernized STL Algorithms

→ Swipe to learn →

C++20 Ranges and Views - Modern Iteration and Transformation

Theory

📌 1. What Are Ranges - The Problem They Solve (1/2)

The Pre-C++20 Problem:

```
// ❌ C++17: Verbose, error-prone, temporary containers
std::vector<int> numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

// Want: even numbers, squared, first 3
std::vector<int> evens;
std::copy_if(numbers.begin(), numbers.end(), std::back_inserter(evens),
             [](int n) { return n % 2 == 0; });

std::vector<int> squared;
std::transform(evens.begin(), evens.end(), std::back_inserter(squared),
               [](int n) { return n * n; });

std::vector<int> result;
std::copy_n(squared.begin(), std::min(3ul, squared.size()),
            std::back_inserter(result));
// Result: {4, 16, 36}
```

Problems:

- 3 intermediate containers (memory overhead)
- Verbose iterator pairs
- Multiple algorithm calls
- Not composable
- Eager evaluation (processes all elements even if you only need a few)

The C++20 Solution:

```
// ✅ C++20: Composable, lazy, zero intermediate containers
auto result = numbers
    | std::views::filter([](int n) { return n % 2 == 0; })
    | std::views::transform([](int n) { return n * n; })
    | std::views::take(3);
// Result: {4, 16, 36} (computed lazily!)

// Convert to container only when needed:
std::vector<int> vec(result.begin(), result.end());
```

C++20 Ranges and Views - Modern Iteration and Transformation

Theory

📌 1. What Are Ranges - The Problem They Solve (2/2)

Benefits:

- **Composable**: Chain operations with `|` operator
- **Lazy**: Computation happens on-demand
- **Zero overhead**: No intermediate containers
- **Readable**: Flows left-to-right like Unix pipes
- **Range-based**: Works with any range (no iterator pairs)

C++20 Ranges and Views - Modern Iteration and Transformation

Theory

📌 2. Range Concepts - The Type Hierarchy (1/2)

C++20 defines a hierarchy of range concepts that specify what operations a range supports:

```
std::ranges::range (has begin/end)
       |
std::ranges::input_range (single-pass read)
       |
std::ranges::forward_range (multi-pass read)
       |
std::ranges::bidirectional_range (can go backwards)
       |
std::ranges::random_access_range (O(1) jump to any element)
       |
std::ranges::contiguous_range (elements in contiguous memory)
```

Detailed Breakdown:

Concept	Requirements	Examples	Can Do
`range`	`begin()`, `end()`	Any container, array	Iterate once
`input_range`	Single-pass iterator	`std::istream_iterator`	Read once
`forward_range`	Multi-pass iterator	`std::forward_list`	Read multiple times
`bidirectional_range`	Bidirectional iterator	`std::list`, `std::set`	Go backwards with `--it`
`random_access_range`	Random access iterator	`std::deque`, `std::vector`	Jump to any position `it + n`
`contiguous_range`	Contiguous memory	`std::vector`, `std::array`, C arrays	Memory is sequential

C++20 Ranges and Views - Modern Iteration and Transformation

Theory

📌 2. Range Concepts - The Type Hierarchy (2/2)

Additional Range Concepts:

```
// Sized range: O(1) size() operation
template<typename R>
concept sized_range = range<R> && requires(R& r) {
    { std::ranges::size(r) } -> std::convertible_to<std::size_t>;
};

// Common range: begin and end have same type
template<typename R>
concept common_range = range<R> &&
    std::same_as<std::ranges::iterator_t<R>, std::ranges::sentinel_t<R>>;

// Borrowed range: iterators remain valid after range destruction
template<typename R>
concept borrowed_range = range<R> && /* implementation-defined */;
```

Checking Range Concepts:

```
#include <ranges>
#include <vector>
#include <list>
#include <forward_list>

std::vector<int> vec;
std::list<int> lst;
std::forward_list<int> fwd;

static_assert(std::ranges::contiguous_range<decltype(vec)>);
static_assert(std::ranges::bidirectional_range<decltype(lst)>);
static_assert(std::ranges::forward_range<decltype(fwd)>);

// All of these are also ranges:
static_assert(std::ranges::range<decltype(vec)>);
static_assert(std::ranges::range<decltype(lst)>);
static_assert(std::ranges::range<decltype(fwd)>);
```

C++20 Ranges and Views - Modern Iteration and Transformation

Theory

📌 3. Views - Lazy, Non-Owning Range Adaptors (1/2)

What is a View?

A view is a lightweight object that:
- **Non-owning**: Doesn't own the data it provides access to
- **Lazy**: Computes elements on-demand (when you iterate)
- **O(1) construction**: Creating a view is cheap (no data copying)
- **Composable**: Views can be chained with `|` operator
- **Movable**: Views are typically move-only (not copyable)

View Concept:

```
template<typename V>
concept view = std::ranges::range<V>
    && std::movable<V>
    && std::ranges::enable_view<V>;
```

Key Difference: View vs Range:

```
std::vector<int> vec{1, 2, 3, 4, 5};  // Range (owns data)

auto v = std::views::filter(vec, [](int n) { return n % 2 == 0; });
// View (doesn't own data, refers to vec)
// If vec is destroyed, v becomes dangling!
```

C++20 Ranges and Views - Modern Iteration and Transformation

Theory

📌 3. Views - Lazy, Non-Owning Range Adaptors (2/2)

Standard Views in C++20:

View	Description	Example
`views::all`	Convert range to view	`views::all(vec)`
`views::filter`	Keep elements matching predicate	`views::filter(is_even)`
`views::transform`	Apply function to each element	`views::transform(square)`
`views::take`	First N elements	`views::take(5)`
`views::take_while`	Elements while predicate true	`views::take_while(is_positive)`
`views::drop`	Skip first N elements	`views::drop(3)`
`views::drop_while`	Skip while predicate true	`views::drop_while(is_negative)`
`views::reverse`	Reverse order	`views::reverse`
`views::elements<N>`	Extract Nth element of tuples	`views::elements<0>`
`views::keys`	Extract keys from pairs	`views::keys`
`views::values`	Extract values from pairs	`views::values`
`views::join`	Flatten nested ranges	`views::join`
`views::split`	Split range by delimiter	`views::split(' ')`
`views::common`	Convert to common_range	`views::common`
`views::counted`	View of N elements from iterator	`views::counted(it, 5)`
`views::iota`	Infinite/bounded sequence	`views::iota(1, 10)`

C++20 Ranges and Views - Modern Iteration and Transformation

Theory

📌 4. Pipeable Syntax - The `|` Operator Magic (1/2)

The Pipe Operator:

C++20 ranges introduce operator overloading for `|` to create readable, left-to-right pipelines:

```
// Traditional function composition (right-to-left, inside-out):
auto result = take(transform(filter(numbers, is_even), square), 3);

// C++20 pipeable syntax (left-to-right, natural reading):
auto result = numbers
    | views::filter(is_even)
    | views::transform(square)
    | views::take(3);
```

How it Works:

```
// Simplified implementation
namespace std::views {
    struct filter_adaptor {
        template<typename Pred>
        auto operator()(Pred pred) const {
            return [pred](auto&& range) {
                return filter_view(std::forward<decltype(range)>(range), pred);
            };
        }
    };

    inline constexpr filter_adaptor filter;
}

// Usage:
auto v1 = numbers | views::filter(is_even);
// Equivalent to:
auto v2 = views::filter(is_even)(numbers);
```

C++20 Ranges and Views - Modern Iteration and Transformation

Theory

📌 4. Pipeable Syntax - The `|` Operator Magic (2/2)

Chaining Multiple Operations:

```
std::vector<std::string> words = {"hello", "world", "cpp", "ranges"};

auto result = words
    | views::filter([](const auto& s) { return s.size() > 3; })  // {"hello", "world", "ranges"}
    | views::transform([](const auto& s) { return s.size(); })    // {5, 5, 6}
    | views::take(2);                                             // {5, 5}

// Lazy! Nothing computed until you iterate:
for (auto len : result) {
    std::cout << len << ' ';  // Output: 5 5
}
```

C++20 Ranges and Views - Modern Iteration and Transformation

Theory

📌 5. Range Algorithms - Modernized STL Algorithms (1/2)

C++20 provides modernized versions of all STL algorithms in `std::ranges` namespace:

Key Improvements Over Traditional Algorithms:

1. **Accept ranges** (not just iterator pairs)
2. **Support projections** (transform before applying)
3. **Return useful results** (not just iterators)
4. **Constrained with concepts** (better error messages)

Comparison:

```
std::vector<int> numbers = {3, 1, 4, 1, 5, 9, 2, 6};

// ❌ C++17: Iterator pairs
std::sort(numbers.begin(), numbers.end());
auto it = std::find(numbers.begin(), numbers.end(), 5);

// ✅ C++20: Ranges
std::ranges::sort(numbers);
auto it = std::ranges::find(numbers, 5);
```

C++20 Ranges and Views - Modern Iteration and Transformation

Theory

📌 5. Range Algorithms - Modernized STL Algorithms (2/2)

Categories of Range Algorithms:

Category	Examples
Non-modifying	`find`, `count`, `all_of`, `any_of`, `none_of`, `for_each`
Modifying	`copy`, `fill`, `replace`, `remove`, `unique`, `reverse`
Sorting	`sort`, `stable_sort`, `partial_sort`, `nth_element`
Binary search	`lower_bound`, `upper_bound`, `binary_search`, `equal_range`
Set operations	`set_union`, `set_intersection`, `set_difference`
Heap operations	`make_heap`, `push_heap`, `pop_heap`, `sort_heap`
Min/max	`min`, `max`, `minmax`, `min_element`, `max_element`
Permutations	`next_permutation`, `prev_permutation`

Return Values:

```
// Traditional: Returns iterator
auto it = std::find(vec.begin(), vec.end(), 5);

// Ranges: Returns iterator + sentinel (subrange)
auto [it, end] = std::ranges::find(vec, 5);

// Some algorithms return more:
auto [min_it, max_it] = std::ranges::minmax_element(vec);

// Some return results:
auto [in_it, out_it] = std::ranges::copy(source, dest.begin());
```

C++20 Ranges and Views - Modern Iteration and Transformation

Theory

📌 6. Projections - Transform Before Applying (1/2)

What is a Projection?

A projection is a callable that transforms an element **before** the algorithm's operation is applied.

Without Projections (C++17):

```
struct Person {
    std::string name;
    int age;
};

std::vector<Person> people = {{"Alice", 30}, {"Bob", 25}, {"Charlie", 35}};

// Sort by age - need to write custom comparator
std::sort(people.begin(), people.end(),
    [](const Person& a, const Person& b) {
        return a.age < b.age;  // Compare ages
    });
```

With Projections (C++20):

```
// Sort by age - just project to age
std::ranges::sort(people, std::less{}, &Person::age);
//                          ^^^^^^^^^^^  ^^^^^^^^^^^^
//                          comparator   projection

// Or with lambda:
std::ranges::sort(people, {}, [](const Person& p) { return p.age; });
```

Projection Parameter:

All range algorithms have this signature pattern:

```
template<std::ranges::range R, typename Proj = std::identity>
void algorithm(R&& r, /* other params */, Proj proj = {});
```

C++20 Ranges and Views - Modern Iteration and Transformation

Theory

📌 6. Projections - Transform Before Applying (2/2)

Common Use Cases:

```
std::vector<std::pair<std::string, int>> pairs = {
    {"apple", 5}, {"banana", 2}, {"cherry", 8}
};

// 1. Sort by second element (value)
std::ranges::sort(pairs, {}, [](const auto& p) { return p.second; });

// 2. Find by first element (key)
auto it = std::ranges::find(pairs, "banana", &std::pair<std::string, int>::first);

// 3. Count elements where second > 5
auto count = std::ranges::count_if(pairs,
    [](int val) { return val > 5; },
    [](const auto& p) { return p.second; }  // projection
);

// 4. Transform using projection
std::vector<std::string> names;
std::ranges::copy(pairs | std::views::transform(&std::pair<std::string, int>::first),
                  std::back_inserter(names));
```

Projection with Views:

```
// Projection in views::filter
auto adults = people
    | views::filter([](int age) { return age >= 18; }, &Person::age);
//                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^  ^^^^^^^^^^^^
//                  predicate operates on int       projection to age
```

C++20 Ranges and Views - Modern Iteration and Transformation

Theory

📌 7. Lazy Evaluation - Compute On-Demand (1/3)

What is Lazy Evaluation?

Views don't compute results when created - they compute elements **only when iterated**.

Demonstration:

```
#include <iostream>
#include <vector>
#include <ranges>

std::vector<int> numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

auto expensive_transform = [](int n) {
    std::cout << "Transforming " << n << '\n';  // Side effect to observe calls
    return n * n;
};

// Creating the view - NO transformations happen yet!
auto view = numbers | std::views::transform(expensive_transform);
std::cout << "View created\n";

// Iterating - transformations happen NOW (on-demand)
for (auto val : view | std::views::take(3)) {
    std::cout << "Got: " << val << '\n';
}

/* Output:
View created
Transforming 1
Got: 1
Transforming 2
Got: 4
Transforming 3
Got: 9
*/
// Only 3 transformations, not 10!
```

C++20 Ranges and Views - Modern Iteration and Transformation

Theory

📌 7. Lazy Evaluation - Compute On-Demand (2/3)

Contrast with Eager Evaluation:

```
// Eager (traditional): Computes all results immediately
std::vector<int> results;
std::transform(numbers.begin(), numbers.end(), std::back_inserter(results),
    expensive_transform);
// All 10 transformations happen here

// Then take first 3
std::vector<int> first_three(results.begin(), results.begin() + 3);

/* Output:
Transforming 1
Transforming 2
Transforming 3
Transforming 4
Transforming 5
Transforming 6
Transforming 7
Transforming 8
Transforming 9
Transforming 10
*/
// Wasted 7 computations!
```

📌 7. Lazy Evaluation - Compute On-Demand (3/3)

Performance Implications:

```
std::vector<int> huge_vector(1'000'000);
std::iota(huge_vector.begin(), huge_vector.end(), 1);

// Lazy: Only processes first 10 elements
auto first_10_evens = huge_vector
    | views::filter([](int n) { return n % 2 == 0; })
    | views::take(10);
// Cost: ~20 iterations to find 10 even numbers

// Eager: Processes all 1,000,000 elements
std::vector<int> all_evens;
std::copy_if(huge_vector.begin(), huge_vector.end(),
    std::back_inserter(all_evens), [](int n) { return n % 2 == 0; });
std::vector<int> first_10(all_evens.begin(), all_evens.begin() + 10);
// Cost: 1,000,000 iterations + memory for 500,000 elements!
```

C++20 Ranges and Views - Modern Iteration and Transformation

Theory

📌 8. Common Range Patterns and Idioms (1/2)

Pattern 1: Filter + Transform + Collect

```
std::vector<int> numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

// Get squares of even numbers
auto result = numbers
    | views::filter([](int n) { return n % 2 == 0; })
    | views::transform([](int n) { return n * n; });

// Materialize to vector
std::vector<int> vec(result.begin(), result.end());
// Or in C++23:
// std::vector<int> vec = result | std::ranges::to<std::vector>();
```

Pattern 2: Infinite Sequences

```
// Generate first 10 Fibonacci numbers
auto fibs = views::iota(0)
    | views::transform([](int n) {
        int a = 0, b = 1;
        for (int i = 0; i < n; ++i) {
            int temp = a + b;
            a = b;
            b = temp;
        }
        return a;
    })
    | views::take(10);
// Lazy! Only computes 10 values even though iota is infinite
```

Pattern 3: Flatten Nested Structures

```
std::vector<std::vector<int>> nested = {{1, 2}, {3, 4, 5}, {6}};

auto flat = nested | views::join;
// Result: {1, 2, 3, 4, 5, 6}
```

C++20 Ranges and Views - Modern Iteration and Transformation

Theory

📌 8. Common Range Patterns and Idioms (2/2)

Pattern 4: Working with Pairs/Tuples

```
std::map<std::string, int> ages = {{"Alice", 30}, {"Bob", 25}};

// Get all names
auto names = ages | views::keys;

// Get all ages > 26
auto adult_ages = ages
    | views::values
    | views::filter([](int age) { return age > 26; });
```

Pattern 5: Split Strings

```
std::string text = "hello world cpp ranges";

auto words = text | views::split(' ');
// Each element is a subrange of the original string

for (auto word : words) {
    std::cout << std::string_view(word.begin(), word.end()) << '\n';
}
```

Pattern 6: Zip (C++23)

```
// C++23 feature
std::vector<int> ids = {1, 2, 3};
std::vector<std::string> names = {"Alice", "Bob", "Charlie"};

for (auto [id, name] : views::zip(ids, names)) {
    std::cout << id << ": " << name << '\n';
}
```

C++20 Ranges and Views - Modern Iteration and Transformation

Theory

📌 9. Custom Views - Creating Your Own Range Adaptors (1/2)

Simple Custom View:

```
// View that repeats each element N times
template<std::ranges::input_range R>
class repeat_view : public std::ranges::view_interface<repeat_view<R>> {
    R base_;
    std::size_t count_;

public:
    repeat_view(R base, std::size_t count)
        : base_(std::move(base)), count_(count) {}

    auto begin() const { return iterator(*this, std::ranges::begin(base_)); }
    auto end() const { return std::ranges::end(base_); }

    class iterator {
        const repeat_view* parent_;
        std::ranges::iterator_t<R> it_;
        std::size_t current_count_ = 0;

    public:
        using value_type = std::ranges::range_value_t<R>;
        using difference_type = std::ptrdiff_t;

        iterator(const repeat_view& parent, std::ranges::iterator_t<R> it)
            : parent_(&parent), it_(it) {}

        decltype(auto) operator*() const { return *it_; }

        iterator& operator++() {
            if (++current_count_ >= parent_->count_) {
                ++it_;
                current_count_ = 0;
            }
            return *this;
        }

        iterator operator++(int) { auto tmp = *this; ++*this; return tmp; }

        bool operator==(const iterator& other) const {
            return it_ == other.it_ && current_count_ == other.current_count_;
// ...continues on next slide
```

C++20 Ranges and Views - Modern Iteration and Transformation

Theory

📌 9. Custom Views - Creating Your Own Range Adaptors (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        }
    };
};

// Deduction guide
template<typename R>
repeat_view(R&&, std::size_t) -> repeat_view<std::views::all_t<R>>;

// Usage:
std::vector<int> v = {1, 2, 3};
repeat_view view(v, 2);
// Result: {1, 1, 2, 2, 3, 3}
```

Range Adaptor Closure Object:

```
// Make it pipeable
namespace views {
    struct repeat_adaptor {
        template<std::ranges::viewable_range R>
        auto operator()(R&& r, std::size_t n) const {
            return repeat_view(std::forward<R>(r), n);
        }

        auto operator()(std::size_t n) const {
            return [n](auto&& r) {
                return repeat_view(std::forward<decltype(r)>(r), n);
            };
        }
    };

    inline constexpr repeat_adaptor repeat;
}

// Now pipeable:
auto result = std::vector{1, 2, 3} | views::repeat(2);
```

C++20 Ranges and Views - Modern Iteration and Transformation

Theory

📌 10. Performance Considerations and Best Practices (1/2)

Best Practices:

1. **Use views for pipelines, materialize when needed:**
   ```cpp
   // Good: Keep as view for further processing
   auto view = data | views::filter(pred) | views::transform(func);

// Materialize only when storing or passing to non-range APIs
std::vector result(view.begin(), view.end());
```

1. **Beware of dangling references:**
   ```cpp
   // ❌ BAD: Temporary vector is destroyed, view dangles
   auto view = std::vector{1, 2, 3} | views::filter(is_even);

// ✅ GOOD: Store the range
auto vec = std::vector{1, 2, 3};
auto view = vec | views::filter(is_even);
```

1. **Use `views::all` for owned ranges:**
   `cpp auto get_view() { std::vector<int> vec{1, 2, 3}; return views::all(std::move(vec)); // View takes ownership }`
2. **Prefer range algorithms over traditional ones:**
   `cpp // Better error messages, cleaner syntax std::ranges::sort(vec); std::ranges::find(vec, 42);`
3. **Use projections instead of custom comparators:**
   `cpp // Clear and concise std::ranges::sort(people, {}, &Person::age);`

C++20 Ranges and Views - Modern Iteration and Transformation

Theory

📌 10. Performance Considerations and Best Practices (2/2)

Performance Gotchas:

```
// ❌ Multiple iterations recompute
auto view = vec | views::transform(expensive_func);
for (auto x : view) { /* ... */ }
for (auto x : view) { /* ... */ }  // Recomputes all!

// ✅ Materialize if iterating multiple times
std::vector<int> computed(view.begin(), view.end());
for (auto x : computed) { /* ... */ }
for (auto x : computed) { /* ... */ }  // No recomputation
```

C++20 Ranges and Views - Modern Iteration and Transformation

Edge Cases

📌 1. Dangling Iterators from Temporary Ranges (1/2)

The Problem:

```
auto get_filtered() {
    return std::vector{1, 2, 3, 4, 5}
        | views::filter([](int n) { return n % 2 == 0; });
}

int main() {
    auto view = get_filtered();

    // ❌ UNDEFINED BEHAVIOR: vector was destroyed after get_filtered() returned
    for (auto x : view) {  // Accessing dangling memory!
        std::cout << x;
    }
}
```

Why It Happens:

- Views are **non-owning** - they reference the underlying range
- The temporary `std::vector` is destroyed after `get_filtered()` returns
- The view holds a dangling reference

Solution 1: Return owned range:

```
auto get_filtered() {
    auto vec = std::vector{1, 2, 3, 4, 5};
    return vec | views::filter([](int n) { return n % 2 == 0; })
               | std::ranges::to<std::vector>();  // C++23
}
```

Solution 2: Use `views::owning_view` (C++23):

```
auto get_filtered() {
    return std::vector{1, 2, 3, 4, 5}
        | views::owning  // Takes ownership of the vector
        | views::filter([](int n) { return n % 2 == 0; });
}
```

Detection:

C++20 has compile-time checks for some dangling cases:

C++20 Ranges and Views - Modern Iteration and Transformation

Edge Cases

📌 1. Dangling Iterators from Temporary Ranges (2/2)

```
// ❌ Compile error in C++20:
auto it = std::ranges::begin(std::vector{1, 2, 3});
// Returns std::ranges::dangling instead of iterator
```

C++20 Ranges and Views - Modern Iteration and Transformation

Edge Cases

📌 2. View Lifetime and Capturing Lambdas

The Problem:

```
auto create_filter(int threshold) {
    // ❌ Lambda captures threshold by reference!
    return [&threshold](int n) { return n > threshold; };
}

int main() {
    auto pred = create_filter(5);  // threshold is on stack
    // create_filter() returns, threshold destroyed

    std::vector<int> vec{1, 3, 7, 9};
    auto view = vec | views::filter(pred);  // Captures dangling reference!

    for (auto x : view) {  // ❌ UNDEFINED BEHAVIOR
        std::cout << x;
    }
}
```

Solution:

```
auto create_filter(int threshold) {
    // ✅ Capture by value
    return [threshold](int n) { return n > threshold; };
}
```

C++20 Ranges and Views - Modern Iteration and Transformation

Edge Cases

📌 3. `views::filter` with Modifying Predicate

The Problem:

```
int count = 0;
auto view = vec | views::filter([&count](int n) {
    ++count;  // Side effect!
    return n % 2 == 0;
});

// ❌ Count is unpredictable!
for (auto x : view) { std::cout << x; }
std::cout << "Count: " << count << '\n';  // How many times was predicate called?

// Iterating again gives different count!
for (auto x : view) { std::cout << x; }
std::cout << "Count: " << count << '\n';  // Count increased!
```

Why It Happens:

- Lazy evaluation means predicate is called during iteration
- Multiple iterations call predicate multiple times
- Stateful predicates lead to non-deterministic behavior

Solution:

Predicates should be pure functions (no side effects).

C++20 Ranges and Views - Modern Iteration and Transformation

Edge Cases

📌 4. Range Concept Requirements and View Composition

The Problem:

```
std::forward_list<int> fwd{1, 2, 3, 4, 5};

// ✅ Works: forward_list supports forward_range
auto v1 = fwd | views::filter([](int n) { return n > 2; });

// ❌ Compile error: reverse requires bidirectional_range
auto v2 = fwd | views::reverse;
// forward_list only provides forward iterators!
```

Type Degradation:

```
std::vector<int> vec{1, 2, 3};  // contiguous_range

auto v1 = vec | views::filter([](int) { return true; });
// v1 is only bidirectional_range (filter can't maintain contiguity)

auto v2 = v1 | views::take(2);
// v2 is also bidirectional_range (inherits weaker category)
```

Rule:

View composition can only maintain or weaken range categories, never strengthen them.

C++20 Ranges and Views - Modern Iteration and Transformation

Edge Cases

📌 5. `views::transform` Return Type Deduction

The Problem:

```
std::vector<int> vec{1, 2, 3};

// ❌ Ambiguous: Lambda returns different types based on input
auto view = vec | views::transform([](int n) {
    if (n % 2 == 0)
        return n;      // returns int
    else
        return n * 1.5;  // returns double
});
// Compilation error: inconsistent return types
```

Solution:

```
// ✅ Explicit return type
auto view = vec | views::transform([](int n) -> double {
    if (n % 2 == 0)
        return n;
    else
        return n * 1.5;
});
```

C++20 Ranges and Views - Modern Iteration and Transformation

Edge Cases

📌 6. `views::split` Produces Views of Views

The Problem:

```
std::string text = "a b c";
auto words = text | views::split(' ');

// ❌ Can't use directly as strings
for (auto word : words) {
    std::cout << word << '\n';  // Compile error: word is a subrange<string::iterator>
}
```

Solution:

```
// ✅ Convert subrange to string_view
for (auto word : text | views::split(' ')) {
    std::string_view sv(word.begin(), word.end());
    std::cout << sv << '\n';
}

// Or transform:
auto string_words = text
    | views::split(' ')
    | views::transform([](auto rng) {
        return std::string_view(rng.begin(), rng.end());
    });
```

C++20 Ranges and Views - Modern Iteration and Transformation

Edge Cases

📌 7. Infinite Ranges and Early Termination

The Problem:

```
// Infinite sequence
auto infinite = views::iota(1);

// ❌ This will never finish!
std::vector<int> vec(infinite.begin(), infinite.end());
```

Solution:

```
// ✅ Limit with take
auto finite = views::iota(1) | views::take(100);
std::vector<int> vec(finite.begin(), finite.end());

// ✅ Or use take_while
auto while_small = views::iota(1)
    | views::take_while([](int n) { return n < 100; });
```

C++20 Ranges and Views - Modern Iteration and Transformation

Code Examples

📌 1. Text Processing Pipeline (1/2)

Problem: Parse log file, extract error messages, count by error code.

Key Techniques:
- Range-based text parsing
- Projection with `&std::pair<int, int>::second`
- `views::take` for limiting results

```
#include <iostream>
#include <fstream>
#include <ranges>
#include <map>
#include <string>
#include <sstream>

namespace views = std::views;

int main() {
    std::ifstream file("server.log");
    std::string line;
    std::map<int, int> error_counts;

    while (std::getline(file, line)) {
        // Pipeline: filter errors → extract code → count
        if (line.find("ERROR") != std::string::npos) {
            // Extract error code (assume format "ERROR [123]: message")
            auto start = line.find('[');
            auto end = line.find(']');
            if (start != std::string::npos && end != std::string::npos) {
                int code = std::stoi(line.substr(start + 1, end - start - 1));
                ++error_counts[code];
            }
        }
    }

    // Find top 5 most common errors
    std::vector<std::pair<int, int>> sorted(error_counts.begin(), error_counts.end());
    std::ranges::sort(sorted, std::greater{}, &std::pair<int, int>::second);

// ...continues on next slide
```

C++20 Ranges and Views - Modern Iteration and Transformation

Code Examples

📌 1. Text Processing Pipeline (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    auto top5 = sorted | views::take(5);

    for (auto [code, count] : top5) {
        std::cout << "Error " << code << ": " << count << " occurrences\n";
    }
}
```

C++20 Ranges and Views - Modern Iteration and Transformation

Code Examples

📌 2. CSV Data Transformation (1/2)

Problem: Read CSV, parse rows, filter by criteria, transform to output format.

```
#include <iostream>
#include <ranges>
#include <string>
#include <vector>
#include <sstream>

namespace views = std::views;

struct Person {
    std::string name;
    int age;
    double salary;
};

Person parse_csv_line(std::string_view line) {
    std::vector<std::string> fields;
    std::string field;
    std::stringstream ss(std::string(line));

    while (std::getline(ss, field, ',')) {
        fields.push_back(field);
    }

    return {fields[0], std::stoi(fields[1]), std::stod(fields[2])};
}

int main() {
    std::vector<std::string> csv_lines = {
        "Alice,30,75000.0",
        "Bob,25,55000.0",
        "Charlie,35,95000.0",
        "Diana,28,65000.0",
        "Eve,32,85000.0"
    };

    // Pipeline: parse → filter age > 28 → sort by salary → format output
    auto high_earners = csv_lines
// ...continues on next slide
```

C++20 Ranges and Views - Modern Iteration and Transformation

Code Examples

📌 2. CSV Data Transformation (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        | views::transform(parse_csv_line)
        | views::filter([](const Person& p) { return p.age > 28; })
        | views::transform([](const Person& p) {
            return std::format("{} (age {}): ${:.2f}", p.name, p.age, p.salary);
        });

    // Sort requires materializing
    std::vector<Person> people;
    for (const auto& line : csv_lines | views::transform(parse_csv_line)) {
        if (line.age > 28) people.push_back(line);
    }
    std::ranges::sort(people, {}, &Person::salary);

    for (const auto& p : people) {
        std::cout << std::format("{} (age {}): ${:.2f}\n", p.name, p.age, p.salary);
    }
}

/* Output:
Charlie (age 35): $95000.00
Eve (age 32): $85000.00
Alice (age 30): $75000.00
*/
```

C++20 Ranges and Views - Modern Iteration and Transformation

Code Examples

📌 3. Graph Traversal with Ranges (1/2)

Problem: BFS traversal using range adapters.

```
#include <iostream>
#include <ranges>
#include <vector>
#include <queue>
#include <unordered_set>

namespace views = std::views;

class Graph {
    std::vector<std::vector<int>> adj_;

public:
    Graph(int n) : adj_(n) {}

    void add_edge(int u, int v) {
        adj_[u].push_back(v);
    }

    auto neighbors(int node) const {
        return adj_[node] | views::all;
    }

    auto bfs(int start) const {
        std::vector<int> result;
        std::queue<int> q;
        std::unordered_set<int> visited;

        q.push(start);
        visited.insert(start);

        while (!q.empty()) {
            int node = q.front();
            q.pop();
            result.push_back(node);

            for (int neighbor : neighbors(node)) {
                if (!visited.contains(neighbor)) {
// ...continues on next slide
```

C++20 Ranges and Views - Modern Iteration and Transformation

Code Examples

📌 3. Graph Traversal with Ranges (2/2)

[Code continues across slides]

```
// ...continued from previous slide
                    visited.insert(neighbor);
                    q.push(neighbor);
                }
            }
        }

        return result;
    }
};

int main() {
    Graph g(7);
    g.add_edge(0, 1);
    g.add_edge(0, 2);
    g.add_edge(1, 3);
    g.add_edge(1, 4);
    g.add_edge(2, 5);
    g.add_edge(2, 6);

    auto traversal = g.bfs(0);

    std::cout << "BFS traversal: ";
    for (int node : traversal) {
        std::cout << node << " ";
    }
    std::cout << '\n';
}

/* Output:
BFS traversal: 0 1 2 3 4 5 6
*/
```

C++20 Ranges and Views - Modern Iteration and Transformation

Code Examples

📌 4. Matrix Operations with Ranges

Problem: Flatten 2D matrix, filter, transform, reshape.

```
#include <iostream>
#include <ranges>
#include <vector>

namespace views = std::views;

int main() {
    std::vector<std::vector<int>> matrix = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };

    // Flatten → filter even → square → collect
    auto result = matrix
        | views::join                                    // Flatten
        | views::filter([](int n) { return n % 2 == 0; })  // Keep even
        | views::transform([](int n) { return n * n; });   // Square

    std::vector<int> vec(result.begin(), result.end());

    std::cout << "Result: ";
    for (int val : vec) {
        std::cout << val << " ";
    }
    std::cout << '\n';
}

/* Output:
Result: 4 16 36 64 100 144
*/
```

C++20 Ranges and Views - Modern Iteration and Transformation

Code Examples

📌 5. Sliding Window with Ranges

Problem: Compute moving average over sliding window.

```
#include <iostream>
#include <ranges>
#include <vector>
#include <numeric>

namespace views = std::views;

template<std::ranges::forward_range R>
auto sliding_window(R&& range, std::size_t window_size) {
    return views::iota(0u, std::ranges::size(range) - window_size + 1)
        | views::transform([&range, window_size](std::size_t i) {
            auto start = std::ranges::begin(range) + i;
            return std::ranges::subrange(start, start + window_size);
        });
}

int main() {
    std::vector<int> data = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    constexpr std::size_t window_size = 3;

    std::cout << "Moving average (window=" << window_size << "):\n";

    for (auto window : sliding_window(data, window_size)) {
        double avg = std::accumulate(window.begin(), window.end(), 0.0) / window_size;
        std::cout << avg << " ";
    }
    std::cout << '\n';
}

/* Output:
Moving average (window=3):
2 3 4 5 6 7 8 9
*/
```

C++20 Ranges and Views - Modern Iteration and Transformation

Code Examples

📌 6. Cartesian Product with Ranges (C++23)

Problem: Generate all combinations of two sequences.

```
#include <iostream>
#include <ranges>
#include <vector>

namespace views = std::views;

int main() {
    std::vector<int> a = {1, 2, 3};
    std::vector<char> b = {'A', 'B'};

    // Cartesian product
    for (int x : a) {
        for (char y : b) {
            std::cout << "(" << x << ", " << y << ") ";
        }
    }
    std::cout << '\n';

    // Using ranges (C++23 views::cartesian_product)
    // for (auto [x, y] : views::cartesian_product(a, b)) {
    //     std::cout << "(" << x << ", " << y << ") ";
    // }
}

/* Output:
(1, A) (1, B) (2, A) (2, B) (3, A) (3, B)
*/
```

C++20 Ranges and Views - Modern Iteration and Transformation

Code Examples

📌 7. Fibonacci Sequence with Ranges

Problem: Generate Fibonacci numbers lazily.

```
#include <iostream>
#include <ranges>

namespace views = std::views;

auto fibonacci() {
    return views::iota(0)
        | views::transform([](int n) {
            int a = 0, b = 1;
            for (int i = 0; i < n; ++i) {
                int next = a + b;
                a = b;
                b = next;
            }
            return a;
        });
}

int main() {
    auto first_20_fibs = fibonacci() | views::take(20);

    std::cout << "First 20 Fibonacci numbers:\n";
    for (int fib : first_20_fibs) {
        std::cout << fib << " ";
    }
    std::cout << '\n';
}

/* Output:
First 20 Fibonacci numbers:
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181
*/
```

C++20 Ranges and Views - Modern Iteration and Transformation

Code Examples

📌 8. Group By with Ranges (1/2)

Problem: Group elements by a key function.

```
#include <iostream>
#include <ranges>
#include <vector>
#include <map>

namespace views = std::views;

template<std::ranges::forward_range R, typename KeyFunc>
auto group_by(R&& range, KeyFunc key_func) {
    using Key = std::invoke_result_t<KeyFunc, std::ranges::range_value_t<R>>;
    std::map<Key, std::vector<std::ranges::range_value_t<R>>> groups;

    for (auto&& elem : range) {
        groups[key_func(elem)].push_back(elem);
    }

    return groups;
}

int main() {
    std::vector<int> numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    // Group by remainder when divided by 3
    auto groups = group_by(numbers, [](int n) { return n % 3; });

    for (const auto& [key, values] : groups) {
        std::cout << "Group " << key << ": ";
        for (int val : values) {
            std::cout << val << " ";
        }
        std::cout << '\n';
    }
}

/* Output:
Group 0: 3 6 9
Group 1: 1 4 7 10
// ...continues on next slide
```

C++20 Ranges and Views - Modern Iteration and Transformation

Code Examples

📌 8. Group By with Ranges (2/2)

[Code continues across slides]

```
// ...continued from previous slide
Group 2: 2 5 8
*/
```

C++20 Ranges and Views - Modern Iteration and Transformation

Code Examples

📌 9. Prime Number Sieve with Ranges

Problem: Generate prime numbers using sieve of Eratosthenes.

```
#include <iostream>
#include <ranges>
#include <vector>

namespace views = std::views;

auto primes_up_to(int n) {
    std::vector<bool> is_prime(n + 1, true);
    is_prime[0] = is_prime[1] = false;

    for (int i = 2; i * i <= n; ++i) {
        if (is_prime[i]) {
            for (int j = i * i; j <= n; j += i) {
                is_prime[j] = false;
            }
        }
    }

    return views::iota(0, n + 1)
        | views::filter([is_prime](int i) { return is_prime[i]; });
}

int main() {
    auto first_10_primes = primes_up_to(100) | views::take(10);

    std::cout << "First 10 primes: ";
    for (int prime : first_10_primes) {
        std::cout << prime << " ";
    }
    std::cout << '\n';
}

/* Output:
First 10 primes: 2 3 5 7 11 13 17 19 23 29
*/
```

C++20 Ranges and Views - Modern Iteration and Transformation

Code Examples

📌 10. JSON-Like Data Processing (1/2)

Problem: Process nested JSON-like structure.

```
#include <iostream>
#include <ranges>
#include <vector>
#include <string>
#include <variant>

namespace views = std::views;

struct User {
    std::string name;
    int age;
    std::vector<std::string> hobbies;
};

int main() {
    std::vector<User> users = {
        {"Alice", 30, {"reading", "gaming", "cooking"}},
        {"Bob", 25, {"gaming", "sports"}},
        {"Charlie", 35, {"reading", "music", "travel"}},
        {"Diana", 28, {"cooking", "travel"}}
    };

    // Find all unique hobbies of users aged > 27
    std::vector<std::string> all_hobbies;

    for (const auto& user : users | views::filter([](const User& u) { return u.age > 27; })) {
        for (const auto& hobby : user.hobbies) {
            all_hobbies.push_back(hobby);
        }
    }

    std::ranges::sort(all_hobbies);
    auto unique_range = std::ranges::unique(all_hobbies);
    all_hobbies.erase(unique_range.begin(), unique_range.end());

    std::cout << "Unique hobbies of users > 27:\n";
    for (const auto& hobby : all_hobbies) {
// ...continues on next slide
```

C++20 Ranges and Views - Modern Iteration and Transformation

Code Examples

📌 10. JSON-Like Data Processing (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        std::cout << "- " << hobby << '\n';
    }
}

/* Output:
Unique hobbies of users > 27:
- cooking
- gaming
- music
- reading
- travel
*/
```

What You Learned

Key Takeaways

- ✓ The Problem They Solve
- ✓ The Type Hierarchy
- ✓ Lazy, Non-Owning Range Adaptors
- ✓ The `|` Operator Magic

→ Swipe to learn →